

TinyTheorem

一个依值类型的编程语言暨定理证明器原型

A prototype for a dependently-typed programming language
with theorem proving

Zhenming Gao¹

¹School of Artificial Intelligence
South China Normal University

April 23, 2026



Challenge & Motivation

Recently:

- LLM & AI4MATH attracted mathematicians such as Terence Tao, circa 2024 $\rightarrow$ PLT, TT, Cat



Challenge & Motivation

Recently:

- LLM & AI4MATH attracted mathematicians such as Terence Tao, circa 2024 $\rightarrow$ PLT, TT, Cat
- React & Vue: Functional programming on the frontend, TypeScript's gradual type system $\rightarrow$ Advanced Functional Programming, PLT



Challenge & Motivation

Recently:

- LLM & AI4MATH attracted mathematicians such as Terence Tao, circa 2024 $\rightarrow$ PLT, TT, Cat
- React & Vue: Functional programming on the frontend, TypeScript's gradual type system $\rightarrow$ Advanced Functional Programming, PLT

Challenge: theory-heavy

PL education and academic (functional) programming practices



Challenge & Motivation

Recently:

- LLM & AI4MATH attracted mathematicians such as Terence Tao, circa 2024 → PLT, TT, Cat
- React & Vue: Functional programming on the frontend, TypeScript's gradual type system → Advanced Functional Programming, PLT

Challenge: theory-heavy

PL education and academic (functional) programming practices

TinyTheorem

solid platform for teaching, experimentation, & verification



TinyTheorem¹ = Tigris + Tigrisd

- a language and toolchain prototype
- aims at industrial programming
 - the *Tigris* programming language
- while simultaneously being theorem proving-oriented
 - the *Tigrisd* theorem prover

Keywords

programming language, type theory, theorem prover, compiler

¹code available at github.com/notch1p/tigris
thesis at github.com/notch1p/beng-thesis



Tigris: A Functional Pure Language

- *pure*: functions in the mathematical sense
- proper functional language: extended λ -calculus as *core calculus*, expressive type system based on Hindley-Milner + Typeclasses



Tigris: A Functional Pure Language

- *pure*: functions in the mathematical sense
- proper functional language: extended λ -calculus as *core calculus*, expressive type system based on Hindley-Milner + Typeclasses
- concise & unified syntax in CAML tradition + heavily influenced by Lean & Haskell with support for custom operators



Tigris: A Functional Pure Language

- *pure*: functions in the mathematical sense
- proper functional language: extended λ -calculus as *core calculus*, expressive type system based on Hindley-Milner + Typeclasses
- concise & unified syntax in CAML tradition + heavily influenced by Lean & Haskell with support for custom operators
- using *Lambda* IR and compiles to Common Lisp (SBCL), enabling FFI
- accessible via compiler CLI frontend & an experimental incremental interpreter



A Glimpse of Tigris

Simple List (linked list) impl

```

type List a = Nil | Cons a (List a)
infixr 67 "::" => Cons
let rec map f
  | Nil => Nil
  | x :: xs => f x :: map f xs
let rec foldl f init
  | Nil => init
  | x :: xs => foldl f (f init x) xs
let hd (x :: _) = x and tl (_ :: xs) = xs

```

The Eq Typeclass

```

class Eq a = {eq : a -> a -> Bool}
instance Eq Int = {eq x y = __eqInt x y}
instance ∀a [Eq a], Eq (Option a) =
{ eq
  | None, None => true
  | Some a, Some b => a = b
  | _, _ => false }

```

Curried & Higher-order fun(app)

```

type Boxed = Boxed (Int -> Int -> Int)
let bAdd = Boxed (_ + _)
let unwrap (Boxed f) = f
let main =
  let (Boxed bAdd') = bAdd in
  (unwrap bAdd 20 20
   , bAdd' 30 30)

```

Operator-extended Let-binding

```

type Option a = None | Some a
infixl 70 "<=>"
let rec xs <=> ys =
  match xs, ys with
  | x :: xs, y :: ys =>
    x + y + xs <=> ys
  | Nil, Nil => 0
  | _, _ => 0
let Some x <*> Some y = Some $ x * y

```



- Parsing: monadic parser combinator + Pratt parsing

²only appear inside constructors.



- Parsing: monadic parser combinator + Pratt parsing
- Type System: Extended HM with typeclass + strict higher-kinded types (HKT) & arbitrary-ranked types², guarded by *skolemization* and careful design, also does System F elaboration

I-ASCRIBE-POLY

$$\begin{array}{c}
\Gamma \vdash e \Rightarrow ty_e; C_e \sigma = \forall \bar{\alpha}. \bar{\pi} \Rightarrow ty \\
\bar{\beta} = \text{fresh}^{|\bar{\alpha}|} \quad \theta = [\bar{\alpha} \mapsto \bar{\beta}] \quad ty_{\text{inst}} = \theta(ty) \\
C_0 = \text{current constraints} \quad (\theta', _) = \text{solve}(C_0, ty_e \sim ty_{\text{inst}}) \\
\mathcal{R} = \text{current rigid set} \quad \forall (\alpha \mapsto ty') \in \theta'. \alpha \in \mathcal{R} \implies ty' = \alpha \\
ty_{\text{sk}} = \text{skolem}(\sigma) \quad \text{add constraints for } \bar{\pi} \\
\hline
\Gamma \vdash (e : \sigma) \Rightarrow ty_{\text{inst}}; C_e, ty_e \sim ty_{\text{inst}}
\end{array}$$

²only appear inside constructors.



- Parsing: monadic parser combinator + Pratt parsing
- Type System: Extended HM with typeclass + strict higher-kinded types (HKT) & arbitrary-ranked types², guarded by *skolemization* and careful design, also does System F elaboration
- IR, CPS, Codegen pipeline: Lambda IR $\xrightarrow{\text{Opt}}$ Closure Conversion $\xrightarrow{\text{Opt}}$ CPS Transformation $\rightarrow$ Common Lisp

I-ASCRIBE-POLY

$$\begin{array}{c}
 \Gamma \vdash e \Rightarrow ty_e; C_e \sigma = \forall \bar{\alpha}. \bar{\pi} \Rightarrow ty \\
 \bar{\beta} = \text{fresh}^{|\bar{\alpha}|} \quad \theta = [\bar{\alpha} \mapsto \bar{\beta}] \quad ty_{\text{inst}} = \theta(ty) \\
 C_0 = \text{current constraints} \quad (\theta', _) = \text{solve}(C_0, ty_e \sim ty_{\text{inst}}) \\
 \mathcal{R} = \text{current rigid set} \quad \forall (\alpha \mapsto ty') \in \theta'. \alpha \in \mathcal{R} \implies ty' = \alpha \\
 ty_{\text{sk}} = \text{skolem}(\sigma) \quad \text{add constraints for } \bar{\pi} \\
 \hline
 \Gamma \vdash (e : \sigma) \Rightarrow ty_{\text{inst}}; C_e, ty_e \sim ty_{\text{inst}}
 \end{array}$$

²only appear inside constructors.



Core Calculus

The type checker runs on:

$expr, e$	$::=$	term
	$const$	constant
	$tname$	variable
	$\lambda x. e$	lambda abstraction
	let $x_1 = e_1$ and .. and $x_n = e_n$ in e'	letexp
	$e e'$	function application
	(e_1, e_2)	product
	if e_1 then e_2 else e_3	conditional
	fix e	fixpoint
	match $e_1, .., e_n$ with $Pmatrix$	pattern matching
	$e : sch$	type ascription



Reconstructing types given by:

$typeexpr, ty, \tau$	$::=$	type expression
	$tyctor$	nullary type constructor
	tv	type variable
	$ty \rightarrow ty'$	arrow type
	$ty \times ty'$	product type
	$tyctor tys$	nominal type application
	κtys	higher-kinded TV head application

- Tigris implements a constraint-solving typechecker, generating constraints throughout the inference process, solving through unification afterwards.



Monad design pattern

```

class Monad (m :  $\lambda$ ) =
  { pure : forall a, a  $\rightarrow$  m a
  , bind :  $\forall$  a b, m a  $\rightarrow$  (a  $\rightarrow$  m b)
     $\rightarrow$  m b }

infixr 90 ">=>" => bind

```

```

instance Monad Option =

```

```

  { pure x = Some x
  , bind
    | None, _ => None
    | Some x, f => f x }

```

```

let main : Option Int =

```

```

  pure 20 >=> fun x =>
    None >=> fun y =>
      pure 30 >=> fun z =>
        pure $ x + y + z

```

Functor design pattern

```

class Functor (m :  $\lambda$ ) =
  { map :  $\forall$  a b, (a  $\rightarrow$  b)  $\rightarrow$  m a
     $\rightarrow$  m b }

infixr 100 "<$>" => map

```

```

instance Functor List =

```

```

  { map f x =
    let rec go
      | Nil => Nil
      | x :: xs =>
        f x :: go xs
    in go f x }

```

```

let main =

```

```

  let k x _ = x in
  ( (2 + _) <$> Some 1
  , k "0" <$> 1 :: 2 :: Nil )

```



mutual

```
type Tree a = Empty | Node a (Forest a)
type Forest a = Nil | Cons (Tree a) (Forest a)
;;
```

let rec countTree

```
| Empty    ⇒ 0
| Node _ f ⇒ 1 + countForest f
```

and countForest

```
| Nil      ⇒ 0
| Cons t f ⇒ countTree t + countForest f
```

(f \$ x = f x, lowest precedence, right associative *)*

let 5 = countTree

```
$ Node 1
$ Cons (Node 2 $ Cons (Node 5 Nil) Nil)
$ Cons (Node 3 $ Cons (Node 4 Nil) Nil)
$ Cons Empty Nil
```



```
type Cont a r = Cont ((a → r) → r) ;; infixr 90 ">>="
```

```
let return x = Cont fun k ⇒ k x
```

```
let run (Cont kf) k = kf k
```

```
let Cont kf >>= f = Cont fun k ⇒
```

```
  kf $ fun a ⇒
```

```
    let (Cont k') = f a
```

```
  in k' k
```

```
let callcc f = Cont fun k ⇒
```

```
  let (Cont kf) = f $ fun a ⇒ Cont fun _ ⇒ k a
```

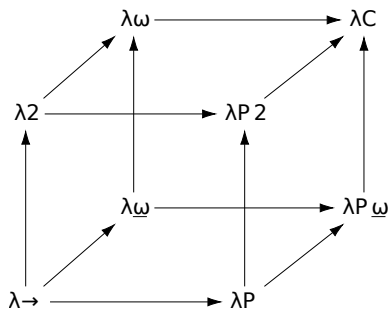
```
  in kf k
```

```
let f = callcc fun k ⇒ return 0 >>= fun _ ⇒ k 42
```

```
let main = run f (fun x ⇒ x)
```



Tigrisδ: A dependently-typed theorem prover

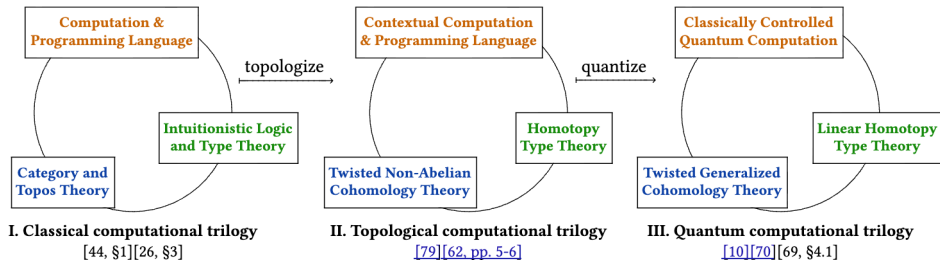


- types may depend on values, types $\in$ terms
- type theory (system) much more expressive than Tigris, intuitionistic (Martin-Löf).
 - Somewhere between $\lambda_{\Pi 2}$ and $\lambda_{\underline{\omega}}$
 - **bare minimum**: dependent type theory + inductive construction
- parser + typechecker/evaluator implementation only



Curry-Howard Isomorphism & Computational Trilogy

- “different perspectives on a single underlying phenomenon at the foundations of mathematics” (per nLab)



C-H Correspondence

Propositions as types; Proofs as terms.

³Sati, Hisham, and Urs Schreiber. "Topological quantum programming in TED-K." arXiv preprint arXiv:2209.08331 (2022)



1	:	Int
$x \mapsto x$	:	$\forall \alpha. \alpha \rightarrow \alpha$
$x \mapsto y$	:	$\forall \alpha, \beta. \alpha \rightarrow \beta$
$x : \alpha \mapsto (h : \beta \ x)$	:	$\forall x : \alpha. \beta \ x := \Pi_{x:\alpha} \beta \ x$
$\langle a : \alpha, h : \beta \ a \rangle$	:	$\exists x : \alpha. \beta \ x := \Sigma_{x:\alpha} \beta \ x$
$\neg p$	$\equiv$	$p \rightarrow \text{False}$
$???$	:	$\forall \alpha. \alpha$
PI-INTRO		SIGMA-INTRO
$\frac{\Gamma, x : \alpha \vdash b : \beta(x)}{\Gamma \vdash \lambda x. b : \Pi_{x:\alpha} \beta(x)}$		$\frac{\Gamma \vdash x : \alpha \quad \Gamma \vdash h : \beta(x)}{\langle x, h \rangle : \Sigma_{x:A} \beta(x)}$

- Intuition: Π, Σ supersedes $\rightarrow, \times$ types, can encode $\forall, \exists$



Indexed (Inductive) Family: Inductive Types

Supercharged

Tigris: ADT

```
type Bool = True | False
type Nat = Zero | Succ Nat
type List a =
  | Nil
  | Cons a (List a)
```

- 1 declaration $\rightarrow$ 1 type

Tigris: dependent GADT (inductive family)

```
inductive Bool : Type | True | False
inductive Fin (n : Nat) : Type
  | Zero (m : Nat) where n = Succ m
  | Succ (m : Nat) ( _ : Fin m) where n = Succ m
inductive Vector (α : Type) (n : Nat) : Type
  | Nil where n = Zero
  | Cons (m : Nat) (x : α) (xs : Vector α m)
    where n = Succ m
```

- 1 declaration $\rightarrow$ a family of types

Directly defining Σ type

```
inductive Sigma (α : Type) (β : α  $\rightarrow$  Type) : Type
  | MkSigma (x : α) (h : β x)
```

- Coinductive family: not possible at this time



Discriminant Refinement

$$\frac{\Gamma \vdash a \Rightarrow A \quad \mathbf{WHNF} \Gamma \vdash A \rightsquigarrow T \bar{c} \quad \Gamma \vdash p : T \bar{c} \Rightarrow \Delta \quad \vdash a \sim p \Rightarrow \Delta' \quad \Gamma, \Delta, \Delta' \vdash b \Leftarrow B}{\Gamma \vdash \text{match}(a, p \rightarrow b) \Leftarrow B} \text{C-MATCH}$$

- case analysis taken literally
- typechecker is aware of the shape of the discriminant in pattern matching branches
- WHNF: a normal form used to control reduction; Crucial for implementing equality. i.e.
 - checking $f(1 + 1) = f\ 2$, preventing full reduction



```

example {α : Type} (n : Nat) (v : Vector α n) : Option α :=
  match n with
  | Zero      ⇒ None
  | Succ Zero ⇒
    match v with
    | Cons _ x Nil ⇒ Some x -- generating h : v = Cons _ x Nil
    | Nil      (...) -- impossible branch, deletable
    | _        ⇒ None

```

inductive Empty

```

example : (C : Type) → Empty → C := fun _ h' ⇒ nomatch h'
-- equivalent to

```

```

def efq {C : Type} (h' : Empty) : C := match h' with

```

- hypothesis h : v must be of length 1
- hypothesis h' : a contradiction, ex falso quodlibet



Typing Rules

$$\boxed{\Gamma \vdash a \stackrel{\text{impl}}{=} b}$$

$$\begin{array}{c} \text{IMPLEQ-REFL} \\ \hline \Gamma \vdash a \stackrel{\text{impl}}{=} a \end{array} \quad \begin{array}{c} \text{IMPLEQ-LAM-CONGR} \\ \hline \Gamma \vdash a_1 \stackrel{\text{impl}}{=} a_2 \\ \hline \Gamma \vdash \lambda x. a_1 \stackrel{\text{impl}}{=} \lambda x. a_2 \end{array}$$

$$\begin{array}{c} \text{IMPLEQ-PI-CONGR} \\ \hline \begin{array}{c} \Gamma \vdash A_1 \stackrel{\text{impl}}{=} A_2 \\ \Gamma \vdash B_1 \stackrel{\text{impl}}{=} B_2 \end{array} \\ \hline \Gamma \vdash (x:A_1) \rightarrow B_{1x} \stackrel{\text{impl}}{=} (x:A_2) \rightarrow B_{2x} \end{array}$$

(equality implementation)

$$\begin{array}{c} \text{IMPLEQ-APP-CONGR} \\ \hline \begin{array}{c} \Gamma \vdash ne_1 \stackrel{\text{impl}}{=} ne_2 \\ \Gamma \vdash b_1 \stackrel{\text{impl}}{=} b_2 \end{array} \\ \hline \Gamma \vdash ne_1 b_1 \stackrel{\text{impl}}{=} ne_2 b_2 \end{array}$$

$$\begin{array}{c} \text{IMPLEQ-WHNF-EXT} \\ \hline \begin{array}{c} \Gamma \vdash \mathbf{WHNF} a \rightsquigarrow nf_1 \\ \Gamma \vdash \mathbf{WHNF} b \rightsquigarrow nf_2 \end{array} \\ \hline \Gamma \vdash nf_1 \stackrel{\text{impl}}{=} nf_2 \\ \hline \Gamma \vdash a \stackrel{\text{impl}}{=} b \end{array}$$



```

( $\Longleftrightarrow$ ) :: Term → Term → FreshMT (ReaderT Env (Except TypingError))()
Var x  $\Longleftrightarrow$  Var y | x = y = return ()
If c t e  $\Longleftrightarrow$  If c' t' e' = c  $\Longleftrightarrow$  c' *> t  $\Longleftrightarrow$  t' *> e  $\Longleftrightarrow$  e'
Let x b1  $\Longleftrightarrow$  Let y b2 =
  b1 `fresh` b2  $\gg=$  \(_, b1, b2) →
    x  $\Longleftrightarrow$  y *> b1  $\Longleftrightarrow$  b2
a  $\Longleftrightarrow$  b | a  $\equiv$  b = return ()

```

```

impleq :: Term → Term → Bi ()
impleq a b = do a ← whnf a; b ← whnf b; liftBi $ a  $\Longleftrightarrow$  b

```

```

liftBi :: FreshMT (ReaderT Env (Except TypingError)) ~> Bi
liftBi (FreshMT{unFreshMT = act}) = FreshMT $
  StateT \st → ReaderT $
    \rst →
      case runExcept $ runReaderT (runStateT act st) rst of
        Left e → throwError e
        Right x → return x

```



• $\text{eqSucc} : \forall m, n \in \mathbb{N}. m = n \rightarrow \text{Succ } m = \text{Succ } n$

def eq_succ {m n : Nat} (h : m = n) : Succ m = Succ n := h ▷ rfl



- $\text{eqSucc} : \forall m, n \in \mathbb{N}. m = n \rightarrow \text{Succ } m = \text{Succ } n$

```
def eq_succ {m n : Nat} (h : m = n) : Succ m = Succ n := h ▸ rfl
```

- $\langle \mathbb{N}, +_{\mathbb{N}} \rangle$ consists of an additive group: $a + b + c = a + (b + c)$

```
def add (x y : Nat) : Nat :=
```

```
  match x with Zero => y | Succ x' => Succ (add x' y)
```

```
def add_assoc (x y z : Nat) : add (add x y) z = add x (add y z) :=
```

```
  match z with
```

```
  | Zero => rfl
```

```
--
```

```
  | Succ z' => eq_succ (add (add x y) z') (add x (add y z')) (add_assoc x y z')
```

$$x + y + z' = x + (y + z')$$



- $\text{eqSucc} : \forall m, n \in \mathbb{N}. m = n \rightarrow \text{Succ } m = \text{Succ } n$

```
def eq_succ {m n : Nat} (h : m = n) : Succ m = Succ n := h ▸ rfl
```

- $\langle \mathbb{N}, +_{\mathbb{N}} \rangle$ consists of an additive group: $a + b + c = a + (b + c)$

```
def add (x y : Nat) : Nat :=
```

```
  match x with Zero => y | Succ x' => Succ (add x' y)
```

```
def add_assoc (x y z : Nat) : add (add x y) z = add x (add y z) :=
```

```
  match z with
```

```
  | Zero => rfl
```

```
--
```

```
  | Succ z' => eq_succ (add (add x y) z') (add x (add y z')) (add_assoc x y z')
```

$$x + y + z' = x + (y + z')$$

- partial order $\leq_{\mathbb{N}}$ is part of the *poset* $\langle \mathbb{N}, \leq_{\mathbb{N}} \rangle$

```
inductive Nat.le (n m : Nat) : Type
```

```
  | refl                                where n = m
```

```
  | step (m' : Nat) (h : Le n m') where m = Succ m'
```



- $\text{eqSucc} : \forall m, n \in \mathbb{N}. m = n \rightarrow \text{Succ } m = \text{Succ } n$

```
def eq_succ {m n : Nat} (h : m = n) : Succ m = Succ n := h ▸ rfl
```

- $\langle \mathbb{N}, +_{\mathbb{N}} \rangle$ consists of an additive group: $a + b + c = a + (b + c)$

```
def add (x y : Nat) : Nat :=
```

```
  match x with
  | Zero => y
  | Succ x' => Succ (add x' y)
```

```
def add_assoc (x y z : Nat) : add (add x y) z = add x (add y z) :=
```

```
  match z with
```

```
  | Zero => rfl
```

```
--
```

```
  | Succ z' => eq_succ (add (add x y) z') (add x (add y z')) (add_assoc x y z')
```

$$x + y + z' = x + (y + z')$$

- partial order $\leq_{\mathbb{N}}$ is part of the *poset* $\langle \mathbb{N}, \leq_{\mathbb{N}} \rangle$

```
inductive Nat.le (n m : Nat) : Type
```

```
  | refl                                where n = m
```

```
  | step (m' : Nat) (h : Le n m') where m = Succ m'
```

- from this we construct the strict partial order $<_{\mathbb{N}}$ given by

```
def Nat.lt (n m : Nat) : Type := Nat.le (Succ n) m
```

that is, $n < m := n + 1 \leq m$.



- Tigrisδ, unlike mainstream provers, is not recursor⁴ based
- we can instead manually derive the recursors for types.

```
def recBool {α : Type}
  (motive : Bool → Type) (brT : motive True) (brF : motive False)
  : (t : Bool) → motive t := fun t ⇒ if t then brT else brF
```

⁴an instruction on how to eliminate/compute on a term of some type, see Part II of the thesis.



- Tigrisid, unlike mainstream provers, is not recursor⁴ based
- we can instead manually derive the recursors for types.

The dependent elimination/computation rule for Bool is given by

BOOL-ELIM

$$\begin{array}{c} \Gamma, x : \text{Bool} \vdash C(x) : \text{Type} \\ \Gamma \vdash p_t : C(\text{True}) \\ \Gamma \vdash p_f : C(\text{False}) \end{array}$$

$$\Gamma \vdash \text{rec}_{\mathbb{B}}(C; p_t; p_f) : \forall b : \text{Bool}. C(b)$$

BOOL-COMP

$$\begin{array}{c} \Gamma, x : \text{Bool} \vdash C(x) : \text{Type} \\ \Gamma \vdash p_t : C(\text{True}) \\ \Gamma \vdash p_f : C(\text{False}) \end{array}$$

$$\begin{array}{l} \Gamma \vdash \text{rec}_{\mathbb{B}}(C; p_t; p_f; \text{False}) \equiv p_f \\ \Gamma \vdash \text{rec}_{\mathbb{B}}(C; p_t; p_f; \text{True}) \equiv p_t \end{array}$$

```
def recBool {α : Type}
  (motive : Bool → Type) (brT : motive True) (brF : motive False)
  : (t : Bool) → motive t := fun t => if t then brT else brF
```

⁴an instruction on how to eliminate/compute on a term of some type, see Part II of the thesis.



- Tigrisdl, unlike mainstream provers, is not recursor⁴ based
- we can instead manually derive the recursors for types.

The dependent elimination/computation rule for Bool is given by

BOOL-ELIM

$$\begin{array}{c} \Gamma, x : \text{Bool} \vdash C(x) : \text{Type} \\ \Gamma \vdash p_t : C(\text{True}) \\ \Gamma \vdash p_f : C(\text{False}) \end{array}$$

$$\Gamma \vdash \text{rec}_{\mathbb{B}}(C; p_t; p_f) : \forall b : \text{Bool}. C(b)$$

BOOL-COMP

$$\begin{array}{c} \Gamma, x : \text{Bool} \vdash C(x) : \text{Type} \\ \Gamma \vdash p_t : C(\text{True}) \\ \Gamma \vdash p_f : C(\text{False}) \end{array}$$

$$\begin{array}{l} \Gamma \vdash \text{rec}_{\mathbb{B}}(C; p_t; p_f; \text{False}) \equiv p_f \\ \Gamma \vdash \text{rec}_{\mathbb{B}}(C; p_t; p_f; \text{True}) \equiv p_t \end{array}$$

def recBool {α : Type}

(motive : Bool → Type) (brT : motive True) (brF : motive False)
 : (t : Bool) → motive t := **fun** t ⇒ **if** t **then** brT **else** brF

- Intuition: Conditionals in provers desugars to recursor application

⁴an instruction on how to eliminate/compute on a term of some type, see Part II of the thesis.



Reflecting on TinyTheorem

Scenario

- teaching functional/effectful programming practices; As an exemplary project for courses such as *Principals of Programming Languages*.
- teaching prover usage/design. Advanced classes may switch to an actual prover such as Coq/Agda/Lean for real-world use.

The Bad & Ugly

- Design flaws such as Lambda IR.
- Theoretically crippled: more academic features such as HKT, Arbitrary-ranked types are not implemented fully/properly. Besides, Tigrisd should probably switch to Calculus of Inductive Constructions to approximate real-world experience.

致谢（排名不分先后）

- ① 邹竞辉老师：在论文编写上指导和把关；
- ② 浙江大学的姚培森老师：在研究方向上给予指导、科研上关照赏识；
- ③ MoonBit 语言开发团队的 leader 张宏波老师：在实习时给予关照，提供 PLDI 工业经验：他们坚定了我继续研究编程语言、类型论的决心；
- ④ 父母、亲人，同宿的李同学、曾同学：前者提供生活上、经济上的支持，在我离校时则有赖后者包办学校各种事务；
- ⑤ 初高中、大学朋友、软件协会、院学生会秘书处的同学们：若非他们愿意陪我唠嗑、作为我的树洞听我吐槽，陪我出去玩，大学生活亦不会如此轻松有趣。

